

System Architecture and Implementation Strategy for the Episdemo Autonomous Agentic Platform

Executive Overview

The paradigm of software application development is undergoing a fundamental shift. The traditional three-tier architecture—comprising a static presentation tier, a rigid application logic tier, and a data tier—is being rapidly superseded by AI-orchestrated stacks. In this new paradigm, the application tier is governed by autonomous agents capable of reasoning, and the presentation tier is generated dynamically in response to real-time context. This report details the comprehensive architectural blueprint for "Episdemo," an automated, plug-and-play platform designed to execute complex, agentic workflows securely.

The Episdemo architecture is engineered to operate efficiently on consumer-grade hardware—specifically baselined against a 16GB RAM Apple M2 Pro MacBook—while orchestrating secure, ephemeral Docker environments per session. It leverages a high-performance Rust core to manage pseudo-terminal (PTY) interactions, dynamically toggle between local and cloud-based Large Language Models (LLMs), and piggyback on installed local AI tools such as the Claude Desktop and Command Line Interface (CLI). Furthermore, Episdemo Abandons manual frontend engineering in favor of AI-driven Generative User Interfaces (GenUI), utilizing React and Shadcn UI to render components like scratchpads and planners on the fly. To bridge the high-performance Rust backend with the dynamic frontend without incurring serialization penalties, the system employs the BoltFFI stack.

This document explores the granular implementation details of each architectural component, providing a synthesized roadmap for building Episdemo securely, efficiently, and at scale.

Core Infrastructure: Ephemeral Orchestration via Rust and Bollard

The bedrock of Episdemo's ability to execute arbitrary terminal commands, run package managers (npm, npx), and initialize project configurations lies in its secure execution environment. Executing AI-generated commands directly on a host machine introduces catastrophic security vulnerabilities, including the risk of prompt-injected malware or destructive file system operations (e.g., `rm -rf`). Consequently, the architecture mandates the orchestration of isolated, ephemeral micro-environments for every active session.

Asynchronous Container Management

To achieve seamless, plug-and-play container orchestration without relying on external cloud microVM services like E2B or Modal, Episdemo integrates the bollard crate into its Rust core. Bollard is an asynchronous Rust client library for the Docker API, leveraging the latest Hyper and Tokio ecosystem improvements to provide a robust, non-blocking interface.

When a user initiates a new Episdemo session, the Rust backend connects to the local Docker daemon. To ensure cross-platform compatibility without manual configuration, the application utilizes Bollard's default socket connection methods, leveraging Unix sockets (/var/run/docker.sock) on macOS/Linux and Named Pipes (\\.pipe/docker_engine) on Windows. The connection strategy is highly configurable via feature flags in the Cargo.toml file, utilizing the pipe feature for minimal binary size or the http and ssl features for remote daemon connections utilizing Rustls.

Upon connection, the system provisions a dedicated container. The configuration of this container is dictated by the ContainerConfig and HostConfig parameters. Because Episdemo frequently requires Node.js initialization, the backend programmatically pulls or builds an image pre-loaded with the necessary runtimes. To execute a dynamic build process, the Rust orchestrator packages the required Dockerfile into a tar archive and streams it to the Docker API using BuildImageOptions and a FramedRead stream.

State Persistence and Volume Sandboxing

While the execution container is ephemeral by design, development environments necessitate the persistence of certain stateful assets, such as node_modules caches or configuration artifacts, to prevent excessive network latency during repeated npm install operations. However, this persistence must not compromise the integrity of the host machine.

Episdemo manages this through highly constrained volume mounting strategies. Using Bollard's HostConfig, the system defines specific volume bindings. To enforce security, Docker-managed volumes are preferred over direct host bind mounts, as Docker-managed volumes are lifecycle-managed by the daemon and decouple the container from the host's directory structure. When bind mounts are strictly necessary for file sharing, the architecture utilizes the nocopy flag to disable automatic data copying from the container path to the volume, preventing the accidental exfiltration of sensitive container state.

Furthermore, the architecture draws inspiration from Kubernetes volume mount policies to enforce strict compliance. The container's root filesystem is mounted with the readonly_rootfs: Option<bool> parameter set to true, excepting the specific working directory allocated for the agent. This guarantees that even if a prompt-injected agent attempts to modify core system binaries or install rootkits, the operation will fail at the filesystem level.

Network Isolation and Egress Filtering

Autonomous agents equipped with terminal access often attempt to download external payloads or contact external APIs. To mitigate the blast radius of a compromised session, Episdemo implements strict network isolation. By default, the container is launched with restricted network capabilities. If initialization commands (e.g., npx create-react-app) require network access, the Rust orchestrator temporarily assigns the container to a bridge network defined within the EndpointIpamConfig. Once the scaffolding is complete and the agent transitions to offline reasoning or file editing, the network mode is programmatically clamped to --network=none, effectively isolating the container from the internet and the host's local network.

Terminal Interaction: Pseudo-Terminals and Subprocess Wrapping

Orchestrating a Docker container is only the first step; Episdemo must actively interact with the processes running inside it. Furthermore, the application is designed to "piggyback" on installed local AI tools, such as the Claude CLI, which expect to run in an interactive terminal. Simply piping strings via standard input (stdin) and standard output (stdout) using Rust's `std::process::Command` is insufficient. Interactive CLIs frequently fail with "Inappropriate ioctl for device" errors on macOS because they detect that they are not attached to a true TeleTYpewriter (TTY).

The PTY Abstraction Layer

To solve this, Episdemo utilizes the `portable-pty` crate, which provides a cross-platform API for pseudo-terminal interfaces. The Rust backend establishes a native PTY system, generating a master/slave PTY pair. The slave end is attached to the target process—whether that is an interactive bash shell spawned inside the Docker container via Bollard's `ExecStartConfig` and `WebSocket` attach features, or a local instance of the Claude CLI spawned as a child process. The orchestrator configures the PTY with a virtual size (e.g., `PtySize { rows: 24, cols: 80, pixel_width: 0, pixel_height: 0 }`), tricking the interactive application into behaving as though a human operator is driving a physical terminal emulator. This preserves ANSI escape sequences, progress bars, and interactive prompts, which are critical for parsing the state of long-running operations like dependency installations.

Expect-Driven Supervision and Interception

Once the PTY is established, Episdemo must parse the unstructured text stream to supervise the agent's actions. The architecture integrates the `rexpect` crate—a Rust port of the classic `expect` library—to automate interactions.

Using non-blocking readers, the Rust backend continuously monitors the PTY output stream. It utilizes regular expressions (e.g., `exp_regex`) and string matching (e.g., `exp_string`) to detect specific operational states, such as a shell prompt (`wait_for_prompt()`), an `npm ERR!`, or a successful build completion.

Crucially, this PTY supervision layer acts as an active security shim. Before the local AI agent is permitted to execute a command within the Docker environment, the proposed command string is routed through an interception mechanism. If the agent attempts to execute unauthorized commands (e.g., global package installations via `apt-get`, or retrieving external scripts via `curl... | bash`), the Rust orchestrator drops the command and injects a mock error message back into the agent's context stream. This technique forces the LLM to rethink its approach without ever exposing the underlying system to the malicious or hallucinated command.

Seamless Integration with Installed AI Tooling

A defining feature of Episdemo is its ability to piggyback on existing, locally installed AI infrastructure, specifically Anthropic's Claude Desktop app or the Claude Code CLI. This approach bypasses the need for the user to supply raw API keys, leveraging their existing authenticated subscriptions (Pro, Team, or Enterprise) via a toggleable runtime.

Headless CLI Wrapping and Remote Control

There are multiple pathways to integrate the Claude Code CLI into Episdemo's architecture. The first involves utilizing the PTY abstraction previously described to run claude as a child process. By wrapping the CLI, Episdemo can programmatically feed specific instructions or grading rubrics into the standard input stream and parse the resulting analysis. If the CLI resists piping, techniques such as using the `--print` flag or leveraging an `LD_PRELOAD` shim (e.g., `libfaketty.so`) can force the CLI to accept programmatic inputs.

However, a more robust architectural pattern leverages Claude Code's native "Remote Control" mode. By executing `claude remote-control` within the project directory, the CLI instantiates a local server session. Episdemo can then connect to this session programmatically, effectively treating the user's local machine as a headless agentic engine. This allows Episdemo to steer in-progress work, pushing context and retrieving generated code without fighting interactive terminal prompts.

Model Context Protocol (MCP) and OAuth Authentication

To deeply integrate Episdemo's localized tools (such as its secure Docker execution environments and generated UIs) with the Claude reasoning engine, the architecture implements the Model Context Protocol (MCP). MCP is a standardized JSON-RPC-based protocol that allows AI models to discover and invoke tools provided by external, local servers. Episdemo's Rust backend runs an embedded MCP server. When the local Claude Desktop or CLI instance is initialized, it reads a configuration file (`.mcp.json`) pointing to the Episdemo MCP endpoint. Consequently, when Claude determines it needs to run an `npm` command or edit a file, it does not attempt to do so directly on the host machine. Instead, it sends an MCP tool execution request to Episdemo. Episdemo validates the request, executes it securely within the ephemeral Docker container, and returns the `stdout/stderr` to Claude.

If Episdemo is deployed in a multi-user or enterprise setting, the MCP server must be secured. The architecture accommodates this by implementing an OAuth 2.0 user consent flow. When the Claude Desktop attempts to connect to the Episdemo MCP server, it is redirected to an authorization URL. Upon user consent, the identity provider returns an authorization code to Claude's callback URLs (`claude.ai` or `claude.com`), which is exchanged for an access token. This ensures that every tool call executed by the AI is strictly authenticated and authorized against the specific user's permissions, preventing privilege escalation.

Hybrid LLM Routing: The Cloud Escape Hatch

While local execution is prioritized for cost reduction, privacy, and offline capabilities, local hardware has strict cognitive limits. Episdemo addresses this by implementing a hybrid routing system with programmable "escape hatches."

The application initially routes tasks to a highly efficient local model (e.g., Qwen 3 4B). The agent attempts to fulfill the user's request within the local Docker sandbox. However, if the local model becomes trapped in an execution loop (e.g., repeatedly failing to parse a complex compilation error) or if the task requires deep, complex reasoning beyond the model's parameter capacity, Episdemo triggers an escape hatch.

The Rust orchestrator monitors the agent's failure rate via the PTY `re[span_52](start_span)[span_52](end_span)xpect` logs. Upon crossing a predefined threshold of consecutive failures, the orchestrator suspends the local inference engine. It then packages the entire context window—including the user's initial prompt, the executed commands, the

resulting error logs, and the local model's failed reasoning steps—and dispatches it to a frontier cloud model, such as Anthropic's Claude 3.5 Sonnet or Google's Gemini, via their respective APIs. The cloud model performs the heavy cognitive lifting, resolves the complex issue, and returns the corrected code or strategy to the local orchestrator, which then resumes local execution. This hybrid approach ensures maximum efficiency without sacrificing the capability to solve highly complex engineering problems.

Local Model Optimization: Overcoming the 16GB Baseline Constraints

The baseline hardware specification for Episdemo is an Apple M2 Pro MacBook with 16GB of Unified Memory. This constraint presents the most significant bottleneck in the architecture. While 16GB is theoretically sufficient for basic tasks, the reality of agentic workflows dictates a highly specific model selection and optimization strategy.

The Unified Memory Bottleneck and "Swap Death"

Apple Silicon's Unified Memory architecture shares RAM between the CPU and the GPU. However, macOS imposes strict limits on the maximum amount of "wired" memory the GPU can allocate, typically capping a 16GB machine at approximately 10.5GB to 11GB of accessible VRAM.

When executing a Large Language Model, the memory footprint consists of the static model weights, the dynamic Key-Value (KV) cache utilized for context window management, and the baseline operating system overhead. For example, a 12B or 14B parameter model (such as Gemma 3 12B) quantized to 4-bit precision requires approximately 8GB of RAM purely for the weights.

In a standard chat interface, this might be manageable. However, Episdemo runs *agentic* workflows. These workflows require massive, persistent context windows containing complex system prompts, JSON schemas for MCP tools, conversation histories, and long terminal outputs. As the agent operates, the KV cache expands rapidly. Once the combined footprint of the weights, the KV cache, and macOS exceeds the physical RAM limit, the operating system aggressively pages memory to the solid-state drive (SSD).

This phenomenon, known as "swap death," causes inference speeds to plummet from a highly interactive 40-60 tokens per second (t/s) down to a grueling 3 t/s. The machine overheats, the UI stutters, and the model's reasoning capabilities degrade as the context window is squeezed. Therefore, to maintain a responsive, human-reading speed (defined as >5 t/s) during deep agentic loops, Episdemo must strictly utilize sub-8B parameter models.

Selecting the Optimal Sub-8B Agentic Model

The current implementation utilizes Qwen 3 4B, which struggles with complex tool invocation. To identify the best alternative local model under 8B parameters, we analyze recent 2026 benchmarking data specifically focused on tool use and execution on M-series hardware using the MLX framework.

Model Designation	Parameter Size	Quantization	Est. RAM Footprint	Inference Speed (M-Series)	Tool Use Accuracy	Architectural Advantage
Qwen 3.5 4B	4B	4-bit (MLX)	~2.4 GB	~158 t/s	73%	Extreme low-latency, allowing for rapid, highly iterative agent loops.
Gemma 3n E4B	~4.9B	4-bit (MLX)	~3.5 GB	~80 t/s	Exceptional	State-of-the-art reasoning for its size, robust tool calling, and multimodal support.
Qwen 3.5 9B	9B	4-bit (MLX)	~5.1 GB	~106 t/s	83%	Stronger native coding, but risks "swap death" when KV cache expands.
Devstral-Small-2	~8B	4-bit (MLX)	~13.4 GB	~47 t/s	17% (Poor)	High RAM footprint relative to performance; unsuitable for this constraint.

Architectural Recommendation: The **Gemma 3n E4B** and the **Qwen 3.5 4B** represent the optimal choices for Episdemo running on a 16GB M2 Pro. The Gemma 3n E4B model punches significantly above its weight class, offering sophisticated reasoning and strict JSON schema adherence comparable to 12B models, making it highly effective for parsing MCP tool payloads. Alternatively, the Qwen 3.5 4B model provides unprecedented throughput (over 150 t/s). This speed is critical for multi-step agentic pipelines where the model must "think," execute a command, read the output, and iterate rapidly before returning a final response to the user.

MLX Framework Integration

To maximize throughput and ensure these models operate efficiently, the Episdemo Rust core bypasses generic CPU runtimes in favor of direct integration with the MLX framework. MLX is an open-source array framework optimized explicitly for Apple Silicon, capable of directly addressing the M2 Pro's Neural Accelerators and dedicated matrix-multiplication operations. By deploying models quantized to 4-bit precision via the mlx-lm library, Episdemo ensures that the model weights and the entire KV cache fit comfortably within the ~10GB GPU wired memory limit, leaving ample RAM for the Rust orchestrator, the Docker container, and the frontend

GenUI to operate smoothly.

Dynamic Few-Shot Prompting for Small Models

While sub-8B models are fast, they natively struggle with complex, zero-shot tool invocation.

They are prone to hallucinating parameters, inventing non-existent tools, or failing to determine the correct execution routing. To compensate for this lack of parameter volume, Episdemo employs an advanced technique known as Dynamic Few-Shot Prompting.

Rather than relying on a monolithic system prompt filled with abstract rules ("always return valid JSON," "never guess IDs"), the architecture maintains a localized, lightweight vector store of successful agent interactions and tool invocations. When a user requests a specific task (e.g., "Install the Stripe SDK and create a basic checkout route"), the backend queries the vector store to retrieve 2 to 3 highly relevant examples of how the agent successfully mapped a similar request into a structured JSON tool parameter in the past.

By injecting these precise, context-aware examples into the prompt payload at runtime, the 4B or 5B model is explicitly guided on exact syntax, routing paths, and reasoning patterns. This dynamic injection drastically hardens the execution pipeline, enabling a tiny, hyper-efficient local model to achieve tool-calling accuracy rates approaching those of massive cloud models, entirely offline and without triggering memory swap.

Generative User Interfaces: Dynamic Frontend Engineering

Traditional software engineering mandates the manual construction of static views, forms, and dashboards. For an AI-native application like Episdemo, which requires dynamic scratchpads, project planners, and terminal output windows, static routing is too rigid. The architecture abandons manual frontend engineering in favor of Generative UI (GenUI)—a paradigm where the AI agent autonomously controls the presentation layer at runtime.

The Component Registry Pattern

Allowing an LLM to emit raw, unstructured HTML or CSS is computationally expensive, prone to visual inconsistencies, and highly susceptible to prompt injection vulnerabilities. Instead, Episdemo implements the "Controlled Generative UI" pattern, often referred to as the Agent-User Interaction (AG-UI) protocol.

The frontend is constructed using React, styled with Tailwind CSS, and utilizes the Shadcn UI design system. Shadcn provides highly modular, accessible, unstyled primitives (via Radix UI) that are customized into a cohesive design language.

The architecture defines a strict registry of these pre-built components within the React application. For example:

Component Identifier	Shadcn UI Elements Utilized	Purpose in Episdemo
ScratchpadComponent	Card, Textarea, ScrollArea	Displays multi-file code generation and iterative edits.
TaskPlannerComponent	Checkbox, Progress, Badge	Renders multi-step agent execution plans and status.
TerminalOutputComponent	Accordion, CodeBlock	Surfaces the PTY stdout/stderr securely from the Docker container.
ApprovalDialogComponent	Dialog, Button (Destructive)	Prompts the user before the agent executes a dangerous

Component Identifier	Shadcn UI Elements Utilized	Purpose in Episdemo
		command.

The Render Lifecycle and JSON Mapping

The orchestration of the UI follows a deterministic lifecycle managed by frameworks like the Vercel AI SDK or LangChain's json-render.

When the user provides a prompt, the local or cloud LLM analyzes the request and invokes a structured tool, outputting a highly constrained JSON payload. Crucially, the LLM does not output UI code. Instead, it outputs a semantic state representation. For example, if the agent creates a project plan, it emits: `{"ui_action": "TaskPlannerComponent", "data": {"tasks": [{"id": 1, "desc": "Init npm"}]}}`.

The React frontend intercepts this JSON payload. A dynamic routing component matches the `ui_action` key to the registered `TaskPlannerComponent`. The structured data within the JSON payload is hydrated into the component as React props, instantly rendering a fully interactive, beautifully styled Shadcn UI element.

This pattern provides a safe, highly performant compromise between flexibility and control. The AI dynamically dictates the interface flow based on real-time context, but the building blocks are strictly defined by the developer, ensuring perfect visual consistency, immediate rendering, and minimal token usage. Furthermore, interactive elements within these generated components (e.g., a user checking off a task in the planner) capture the interaction and translate it back into structured tool calls directed at the AI agent, creating a seamless, bidirectional data loop.

High-Performance FFI: Integrating the Rust and Bolt Stack

Episdemo consists of two distinct operational halves: the heavily optimized Rust core (managing Docker, PTY, MCP, and local model routing) and the dynamic presentation layer (React, JavaScript/TypeScript, and Shadcn). Establishing rapid, synchronous communication between these layers requires a Foreign Function Interface (FFI).

Historically, tools like UniFFI or wasmbindgen have been the standard for generating these cross-language bindings. However, these traditional FFI tools introduce severe performance bottlenecks in high-throughput applications. UniFFI operates via serialization, converting every value and struct into a byte buffer before passing it across the boundary. Similarly, wasmbindgen materializes every Rust struct as an entirely new JavaScript object. When an AI agent streams hundreds of tokens per second (e.g., Qwen 3.5 4B at 158 t/s), or a PTY buffer streams large volumes of terminal compilation output, this constant serialization and deserialization overhead cascades into noticeable UI latency, garbage collection pauses, and CPU spikes.

Zero-Copy Architecture via BoltFFI

To eliminate this boundary overhead, Episdemo mandates the use of BoltFFI. BoltFFI is a high-performance bindings generator engineered specifically around a zero-copy philosophy. When the Rust orchestrator needs to pass data to the React frontend, BoltFFI ensures that primitive types are passed as raw values. For complex structures containing primitive fields (such as agent state tracking or UI component payloads), the data is passed as pointers to

memory that both the Rust environment and the host environment can read directly, circumventing the serialization phase entirely. Benchmarks indicate that BoltFFI operations are over 1,000x faster than UniFFI for basic calls, and drastically more efficient for complex struct instantiation.

Implementation and Multi-Target Packaging

Integrating BoltFFI into the Episdemo Rust stack is declarative and highly ergonomic. Core Rust structures and orchestration functions intended for the frontend are simply annotated with specific macros:

```
use boltffi::*;

#[data]
pub struct AgentState {
    pub session_id: u64,
    pub active_tokens: u32,
    pub current_ui_action: String,
}

#[export]
pub fn execute_mcp_tool(state: AgentState, payload: String) -> bool {
    // Rust Bollard and PTY execution logic here...
}
```

The build pipeline leverages the boltffi pack command to generate artifacts across multiple deployment targets simultaneously.

- If Episdemo is compiled as a web application or an Electron wrapper, running boltffi pack wasm generates the .wasm binaries alongside deeply typed TypeScript definitions and a ready-to-use npm package. The WASM wire buffer format is uniquely tuned to avoid per-field allocation overhead.
- If Episdemo is deployed as a native macOS binary (e.g., via Tauri or native Swift wrappers), boltffi pack apple produces a highly optimized .xcframework. The architecture configuration (boltffi.toml) allows for precise ABI matrix selection. The developer specifies macos_architectures = ["arm64"] to ensure the binary is compiled explicitly for the M2 Pro chip, stripping unnecessary x86 cross-compilation bloat and reducing the application footprint.

This integration ensures that the complex state machines, Docker orchestration loops, and PTY stream parsers running in the Rust core are instantly available to the React/Shadow generative UI layer without dropped frames or performance degradation.

Architectural Security Synthesis

The convergence of autonomous LLM reasoning, dynamic local code execution, and programmatic terminal access requires a rigid, defense-in-depth security posture. An agent instructed to "clean up the project repository" might hallucinate and execute a system-wide deletion command. Episdemo mitigates these risks across every tier of the architecture:

1. **Micro-Environment Isolation:** By routing all executable tool calls into Bollard-managed

ephemeral Docker containers, the host filesystem is physically isolated from the agent's logic loop. Catastrophic commands only destroy a disposable environment that takes milliseconds to recreate.

2. **Filesystem & Egress Locks:** The Docker containers utilize the `--network=none` flag to block external API calls by default, and a `readonly_rootfs` to protect underlying container binaries from modification.
3. **Command Interception (PTY Shim Layer):** The rexpct supervision logic traps dangerous package manager calls before they are executed in the shell, requiring explicit user approval via the Generative UI before proceeding.
4. **Least Privilege MCP Execution:** The embedded Model Context Protocol server strictly limits the tools exposed to the LLM. The AI is granted only the minimal set of capabilities required to fulfill the immediate task, dramatically reducing the attack surface of prompt injection payloads.
5. **OAuth Boundaries:** When piggybacking on external tools like the Claude Desktop, the MCP connection is secured via standard OAuth 2.0 flows, guaranteeing that all agent actions are explicitly authorized by the user.

By synthesizing a high-performance Rust core, zero-copy BoltFFI bindings, MLX-optimized local AI inference, and dynamic React-driven Generative UIs, the Episdemo architecture delivers a secure, highly responsive, and fully autonomous platform capable of sophisticated software engineering tasks while operating cleanly within the strict limitations of baseline consumer hardware.

Works cited

1. What Is Three-Tier Architecture? | IBM, <https://www.ibm.com/think/topics/three-tier-architecture>
2. Understanding the architecture of a 3-tier application - vFunction, <https://vfunction.com/blog/3-tier-application/>
3. The Three Layers of the AI Stack | AI Engineering - AlgoMaster.io, <https://algomaster.io/learn/ai-engineering/the-three-layers-of-the-ai-stack>
4. Layers of the AI Stack, Explained Simply | Towards Data Science, <https://towardsdatascience.com/layers-ai-stack/>
5. Fine tune LLM on 16GB Macbook M2 Pro using MLX | by Debby Lee | Medium, <https://medium.com/@debby1ee/fine-tune-llm-on-16gb-macbook-m2-pro-using-mlx-06b47047bc1d>
6. Gemma4 26B A4B runs easily on 16GB Macs : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1scjoox/gemma4_26b_a4b_runs_easily_on_16gb_macs/
7. Continue local sessions from any device with Remote Control - Claude Code Docs, <https://code.claude.com/docs/en/remote-control>
8. A Rust Local AI CLI Project: Another Path for Ollama, Small Models, and Developer Tools, <https://users.rust-lang.org/t/a-rust-local-ai-cli-project-another-path-for-ollama-small-models-and-developer-tools/139569>
9. Directed Context Programming for AI-Generated UIs | by Peter ..., https://medium.com/@peter_84200/directed-context-programming-for-ai-generated-uis-588505a1172b
10. How to implement generative user interfaces with LangGraph - Docs by LangChain, <https://docs.langchain.com/langsmith/generative-ui-react>
11. BoltFFI: a high-performance Rust bindings generator (up to 1000x vs UniFFI microbenchmarks) - Reddit, https://www.reddit.com/r/rust/comments/1r768bm/boltffi_a_highperformance_rust_bindings_generator/
12. boltffi/boltffi: A high-performance multi-language bindings ... - GitHub, <https://github.com/boltffi/boltffi>
13. Docker Sandboxes: Run Agents in YOLO Mode, Safely, <https://www.docker.com/blog/docker-sandboxes-run-agents-in-yolo-mode-safely/>
14. Built a CLI

tool that sandboxes everything AI agents try to install on your machine. - Reddit, https://www.reddit.com/r/ClaudeCode/comments/1rj2kt6/built_a_cli_tool_that_sandboxes_everything_ai/ 15. AI Agent Sandbox: How to Safely Run Autonomous Agents in 2026 - Firecrawl, <https://www.firecrawl.dev/blog/ai-agent-sandbox> 16. E2B Documentation, <https://e2b.dev/docs> 17. bollard - Rust - Docs.rs, <https://docs.rs/bollard> 18. bollard-containerd - crates.io: Rust Package Registry, <https://crates.io/crates/bollard-containerd> 19. bollard::service - Rust, <https://wiki.enablingpersonalizedinterventions.nl/docs/bollard/service/index.html> 20. HostConfig in bollard::models - Rust - Docs.rs, <https://docs.rs/bollard/latest/bollard/models/struct.HostConfig.html> 21. Can someone familiar with Bollard, a docker library for rust, help me with it? I have a problem I can't solve. - Reddit, https://www.reddit.com/r/rust/comments/lfpj4f/can_someone_familiar_with_bollard_a_docker/ 22. EP 16 - Docker Storage Explained: Ephemeral Containers, Persistent Data (Volumes vs Bind Mounts) - YouTube, <https://www.youtube.com/watch?v=uAzMFy1bXbQ> 23. Securing the usage of volumeMounts - Kubewarden, <https://www.kubewarden.io/blog/2022/11/volumemounts-policy/> 24. Use Claude Code in VS Code - Claude Code Docs, <https://code.claude.com/docs/en/vs-code> 25. How to create a wrapper for interactive terminal commands in Rust? - Reddit, https://www.reddit.com/r/rust/comments/1djlfpc/how_to_create_a_wrapper_for_interactive_terminal/ 26. portable_pty - Rust - Docs.rs, <https://docs.rs/portable-pty> 27. rexpect - crates.io: Rust Package Registry, <https://crates.io/crates/rexpect> 28. rexpect - Rust - Docs.rs, <https://docs.rs/rexpect> 29. How can I call to Claude in the CLI AS IF it were an API? : r/ClaudeCode - Reddit, https://www.reddit.com/r/ClaudeCode/comments/1qsjfzr/how_can_i_call_to_claude_in_the_cli_as_if_it_were/ 30. Calling Claude CLI as a child process yields no output - Stack Overflow, <https://stackoverflow.com/questions/79826420/calling-claude-cli-as-a-child-process-yields-no-output> 31. How to fool a subprocess into thinking its stdout/stderr was a TTY while still reading output of ... - Rust Users Forum, <https://users.rust-lang.org/t/how-to-fool-a-subprocess-into-thinking-its-stdout-stderr-was-a-tty-while-still-reading-output-of-its-stdout-stderr/79810> 32. CLI reference - Claude Code Docs, <https://code.claude.com/docs/en/cli-reference> 33. Platforms and integrations - Claude Code Docs, <https://code.claude.com/docs/en/platforms> 34. Connect Claude Code to tools via MCP, <https://code.claude.com/docs/en/mcp> 35. MCP Tool-Use with Ollama to Empower Your Local AI Agents | by Brendan Copley - Medium, <https://medium.com/renaissance-learning-r-d/mcp-tool-use-with-ollama-to-empower-your-local-ai-agents-1f12df974982> 36. Claude Connector Authentication: How OAuth Works and When You Need It - Sunpeak.ai, <https://sunpeak.ai/blogs/claude-connector-oauth-authentication/> 37. Build MCP Connector with OAuth for Claude Desktop | Python Tutorial - YouTube, <https://www.youtube.com/watch?v=gdFn5ul1Ool> 38. Mac Mini M4 Pro 24GB - local LLMs are unusable for real work. Would clustering a second one help? : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1rif3h5/mac_mini_m4_pro_24gb_local_llms_are_unusable_for/ 39. Stop Running Models Too Big for Your Mac (Memory Trap Explained) - YouTube, <https://www.youtube.com/shorts/vdkMNY5LPho> 40. Run AI Locally: The Best LLMs for 8GB, 16GB, 32GB Memory and Beyond - Micro Center, <https://www.microcenter.com/site/mc-news/article/best-local-llms-8gb-16gb-32gb-memory-guide.aspx> 41. 16GB is the new 8GB. Why my "Pro" workflow hit a wall (and why I'm jumping to 48GB) : r/macbookpro - Reddit, https://www.reddit.com/r/macbookpro/comments/1quspeg/16gb_is_the_new_8gb_why_my_pro_workflow_hit_a/ 42. Running LLMs Locally in 2025: Speed tests on M2 Pro + 16 GB ...

<https://adamjones.me/blog/local-llms-speed-early-2025/> 43. Benchmarked 11 MLX models on M3 Ultra — here's which ones are actually smart and fast, https://www.reddit.com/r/LocalLLaMA/comments/1rkcvqa/benchmarked_11_mlx_models_on_m3_ultra_heres_which/ 44. Which are the top LLMs under 8B right now? : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1qcl543/which_are_the_top_llms_under_8b_right_now/ 45. Exploring LLMs with MLX and the Neural Accelerators in the M5 GPU, <https://machinelearning.apple.com/research/exploring-llms-mlx-m5> 46. Few-Shot Prompting for Agentic Systems: Teaching by Example - Comet, <https://www.comet.com/site/blog/few-shot-prompting/> 47. Few-shot prompting to improve tool-calling performance - LangChain, <https://www.langchain.com/blog/few-shot-prompting-to-improve-tool-calling-performance> 48. Optimizing AI Agents with Dynamic Few-Shot Prompting - Medium, <https://medium.com/@stefansipinkoski/optimizing-ai-agents-with-dynamic-few-shot-prompting-585919f694cc> 49. Generative UI examples for: AG-UI, A2UI/Open-JSON-UI, and MCP Apps. - GitHub, <https://github.com/CopilotKit/generative-ui> 50. The Developer's Guide to Generative UI in 2026 | Blog - CopilotKit, <https://www.copilotkit.ai/blog/the-developer-s-guide-to-generative-ui-in-2026> 51. Shadcn UI components vs AI-generated components with Tailwind css : r/reactjs - Reddit, https://www.reddit.com/r/reactjs/comments/1r8ikab/shadcn_ui_components_vs_aigenerated_components/ 52. The AI-Native shadcn/ui Component Library for React, <https://www.shadcn.io/> 53. narrowin/awesome-generative-ui: A curated list of resources for AI-generated user interfaces — systems where LLMs dynamically create, compose, and render UI components. - GitHub, <https://github.com/narrowin/awesome-generative-ui> 54. Generative UI - Docs by LangChain, <https://docs.langchain.com/oss/python/langchain/frontend/generative-ui> 55. Building a UI in 60 seconds with Shadcn and Framer AI - LogRocket Blog, <https://blog.logrocket.com/building-ui-60-seconds-shadcn-framer-ai/> 56. Learnings from building multiple Generative UI agents, here's what I learned : r/LangChain, https://www.reddit.com/r/LangChain/comments/1klw9z/learnings_from_building_multiple_generative_ui/ 57. The UniFFI user guide, <https://mozilla.github.io/uniffi-rs/>